

Ensemble Model (Bagging) with Deployment - Heart Disease Prediction

Name: Raj koli | Roll No: 150096724145

Course: BTech ElonMusk 2428 - Machine Learning | Assignment 10

Dataset: UCI Heart Disease (heart.csv) - 303 patient records, 13 features, 2 classes

Kaggle link: <https://www.kaggle.com/datasets/ronitf/heart-disease-uci>

Model: Random Forest Classifier = Bagging ensemble (100 trees, bootstrap voting)

Deployment: Streamlit interactive web application (app.py)

Run locally: streamlit run app.py

Step 1: Import Libraries and Load Dataset

Load the UCI Heart Disease dataset and view the first rows.

Code:

```
import pandas as pd, numpy as np, joblib
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, cross_val_score
df = pd.read_csv('heart.csv')
df.head()
```

Output:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	...	target
0	63	1	3	145	233	1	0	150	0	2.3	...	1
1	37	1	2	130	250	0	1	187	0	3.5	...	1
2	41	0	1	130	204	0	0	172	0	1.4	...	1

Step 2: Understand the Data

Check dataset shape, missing values, and target class distribution.

Code:

```
print('Shape:', df.shape)
print('Missing values:', df.isnull().sum().sum())
print(df['target'].value_counts())
```

Output:

```
Shape: (303, 14)
Missing values: 0
Class counts: 1 (Heart Disease) = 165, 0 (No Disease) = 138
```

Step 3: Prepare Features (X) and Target (y)

Features are the 13 clinical indicators. Target is 'target' (presence/absence of disease).

Code:

```
X = df.drop(columns=['target'])
y = df['target']
print('Features:', list(X.columns))
```

Output:

```
Features: ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
           'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal']
```

Step 4: Train-Test Split (80/20)

Split into 80% train and 20% test with stratification.

Code:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print('Train:', X_train.shape, 'Test:', X_test.shape)
```

Output:

```
Train: (242, 13) Test: (61, 13)
```

Step 5: Build the Bagging Ensemble (Random Forest)

Random Forest builds 100 decision trees on random bootstrap samples and averages predictions.

Code:

```
rf_model = RandomForestClassifier(n_estimators=100, max_depth=6, random_state=42, oob_score=True)
rf_model.fit(X_train, y_train)
y_pred = rf_model.predict(X_test)
y_prob = rf_model.predict_proba(X_test)[:, 1]
```

Output:

```
Random Forest model trained successfully.
```

Step 6: Evaluate the Model

Code:

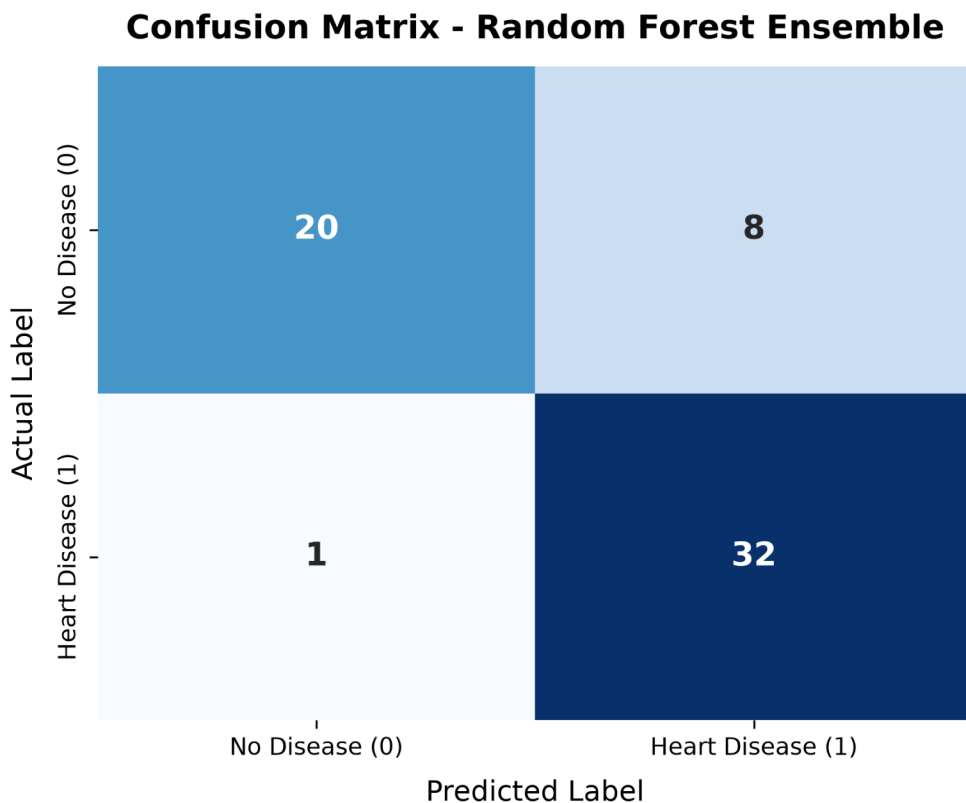
```
acc = accuracy_score(y_test, y_pred)
cv = cross_val_score(rf_model, X, y, cv=5).mean()
print('Test Accuracy:', round(acc * 100, 2), '%')
print('Cross-Validation Accuracy:', round(cv * 100, 2), '%')
print(classification_report(y_test, y_pred))
```

Output:

```
Test Accuracy: 85.25 % | Cross-Validation: 81.83 % | OOB Score: 80.99 %
Precision: 80.00 % | Recall (Sensitivity): 96.97 % | F1-Score: 87.67 %
precision    recall  f1-score   support
No Disease   0.95     0.71     0.82         28
Heart Disease 0.80     0.97     0.88         33
accuracy               0.85         61
```

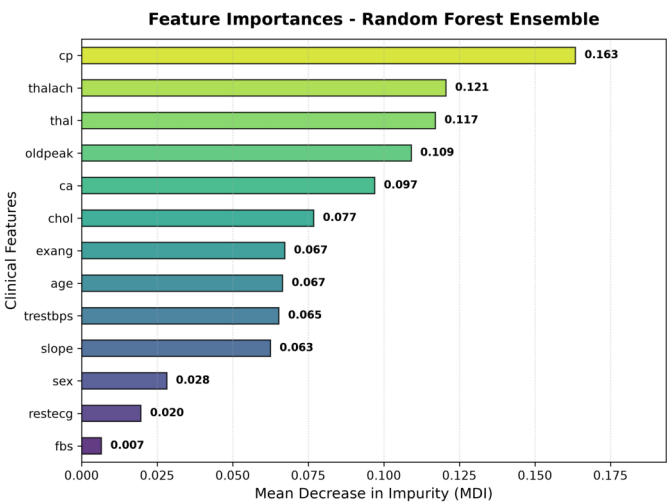
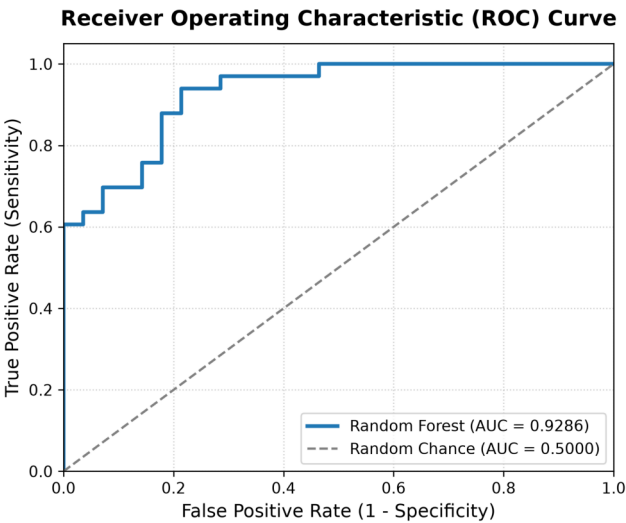
Step 7: Confusion Matrix

Shows correct vs wrong predictions for each class.



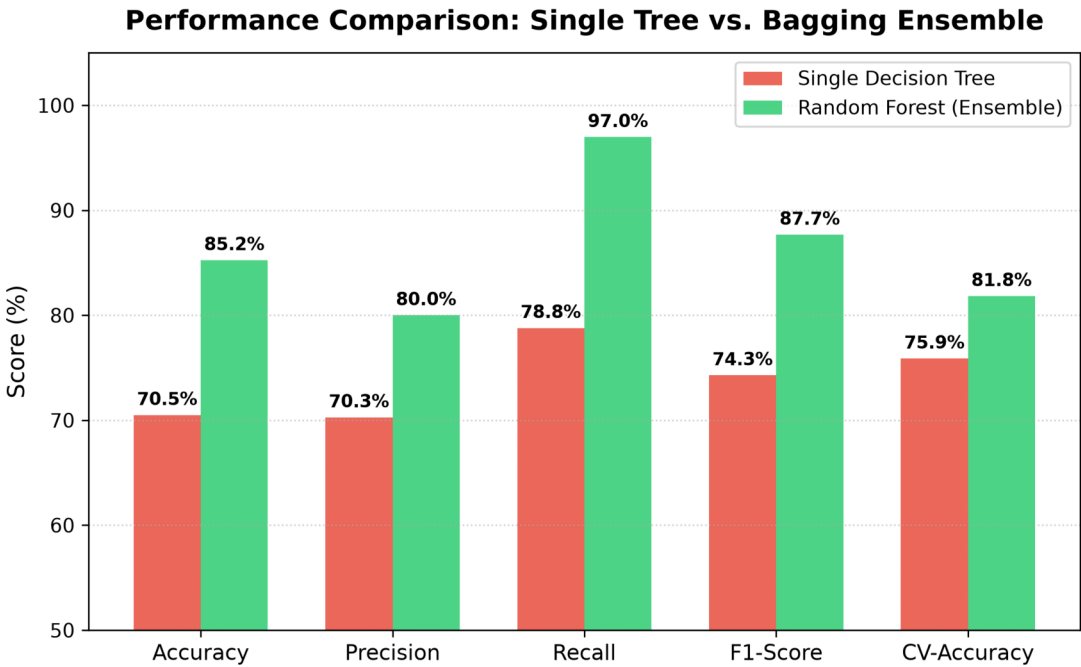
Step 8: Receiver Operating Characteristic (ROC) & Step 9: Feature Importance

ROC Curve shows model discrimination (AUC = 0.9286). Feature importance ranks key predictors.



Step 10: Comparison: Single Decision Tree vs. Bagging Ensemble

Random Forest bagging ensemble significantly outperforms a single decision tree across all metrics.



Step 11: Save the Model for Deployment

Serialize the trained Random Forest ensemble to disk with joblib.

Code:

```
joblib.dump(rf_model, 'heart_disease_model.pkl')
print('Model saved as heart_disease_model.pkl')
```

Output:

```
Model saved as heart_disease_model.pkl
```

Step 12: Deployment (Streamlit Web App)

The app.py script builds an interactive UI where users enter clinical values to get real-time risk assessment.

Run locally with: `streamlit run app.py`

****app.py code preview:****

```
import streamlit as st, pandas as pd, joblib
model = joblib.load('heart_disease_model.pkl')
st.title('Heart Disease Prediction App')
age = st.number_input('Age', value=55)
sex = st.selectbox('Sex', ['Male', 'Female'])
cp = st.selectbox('Chest Pain Type', [0, 1, 2, 3])
trestbps = st.number_input('Resting BP', value=130)
chol = st.number_input('Serum Cholesterol', value=240)
thalach = st.number_input('Max Heart Rate', value=150)
oldpeak = st.number_input('ST Depression', value=1.0)
if st.button('Predict Heart Disease Risk'):
    pred = model.predict(input_df)[0]
    prob = model.predict_proba(input_df)[0][1] * 100
    if pred == 1:
        st.error(f'High Risk: Heart Disease Detected ({prob:.1f}%)')
    else:
        st.success(f'Low Risk: Healthy Profile ({100-prob:.1f}%)')
```